

Actividad – Pruebas Automatizadas de Software

1.1 Define con tus propias palabras

¿Qué son las pruebas unitarias?

Las pruebas unitarias son pruebas que verifican el funcionamiento de una pequeña parte del código, como una función o método, de manera aislada. Su objetivo es comprobar que cada componente funcione correctamente antes de integrarlo con otros módulos.

Ejemplo: verificar que una función de suma retorne el resultado correcto.

¿Cómo difieren de las pruebas de integración?

Las pruebas de integración validan que varios módulos o componentes trabajen correctamente juntos.

Mientras las pruebas unitarias prueban partes pequeñas e individuales, las de integración comprueban la comunicación entre bases de datos, APIs, servicios o módulos completos.

Ejemplo: comprobar que un formulario guarde correctamente datos en la base de datos.

¿Qué son las pruebas E2E?

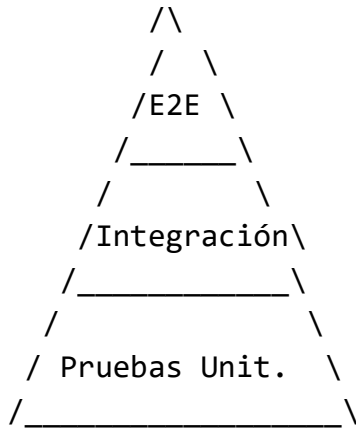
Las pruebas E2E (End to End) simulan el comportamiento real del usuario dentro de toda la aplicación.

Estas pruebas verifican flujos completos desde el inicio hasta el final.

Ejemplo: un usuario inicia sesión, agrega un producto al carrito y realiza una compra exitosamente.

1.2 Explica la Pirámide de Pruebas

Pirámide de Pruebas



Explicación de cada nivel

1. Pruebas Unitarias (Base)

Son la mayor cantidad de pruebas.

Verifican funciones individuales de manera rápida y aislada.

Características:

- Muy rápidas
- Fáciles de ejecutar
- Detectan errores temprano
- Bajo costo de mantenimiento

2. Pruebas de Integración (Nivel Medio)

Comprueban que diferentes módulos funcionen correctamente juntos.

Características:

- Velocidad media
- Validan conexiones entre componentes
- Detectan errores de comunicación

3. Pruebas E2E (Cima)

Validan el flujo completo de la aplicación simulando usuarios reales.

Características:

- Más lentas
- Más complejas
- Más frágiles
- Menor cantidad

¿Por qué es importante esta distribución?

La pirámide permite tener:

- Mayor velocidad en las pruebas
- Menor costo de mantenimiento
- Detección temprana de errores
- Mejor estabilidad en el proyecto

La mayoría de pruebas deben ser unitarias porque son rápidas y fáciles de mantener.

¿Qué problemas surgen cuando la pirámide se invierte?

Cuando hay demasiadas pruebas E2E y pocas unitarias ocurren problemas como:

- Ejecución lenta
- Mayor consumo de recursos
- Difícil mantenimiento
- Más fallos falsos
- Desarrollo más lento
- Mayor dificultad para encontrar errores

1.3 Investigación sobre TDD (Test-Driven Development)

¿Qué es TDD?

TDD es una metodología donde primero se escriben las pruebas y luego el código de la aplicación.

Ciclo Red – Green – Refactor

Red

Se crea una prueba que inicialmente falla porque la funcionalidad aún no existe.

Green

Se escribe el código mínimo necesario para que la prueba funcione correctamente.

Refactor

Se mejora y optimiza el código sin cambiar su comportamiento.

Beneficios principales de TDD

- Código más limpio y organizado
- Menos errores en producción
- Mejor diseño del software
- Facilita el mantenimiento
- Mayor confianza al modificar código

2 desafíos comunes al implementar TDD

1. Curva de aprendizaje

Al principio puede ser difícil acostumbrarse a escribir pruebas antes del código.

2. Mayor tiempo inicial

El desarrollo puede parecer más lento al inicio porque primero se crean las pruebas.

2. Investigación de Herramientas y Configuración

Node.js

Herramientas populares:

- Jest
- Mocha
- Cypress

Uso:

Permiten realizar pruebas unitarias, integración y E2E en aplicaciones JavaScript.

PHP

Herramientas populares:

- PHPUnit
- PestPHP

Uso:

Sirven para probar aplicaciones PHP y frameworks como Laravel.

Java

Herramientas populares:

- JUnit
- TestNG
- Selenium

Uso:

Se utilizan para pruebas automatizadas en aplicaciones Java empresariales y web.

Conclusión

Las pruebas automatizadas son fundamentales para garantizar la calidad del software. Gracias a ellas es posible detectar errores rápidamente, mejorar la estabilidad de las aplicaciones y acelerar el desarrollo. La pirámide de pruebas ayuda a organizar correctamente los diferentes tipos de testing, mientras que metodologías como TDD fomentan un código más limpio y confiable.